

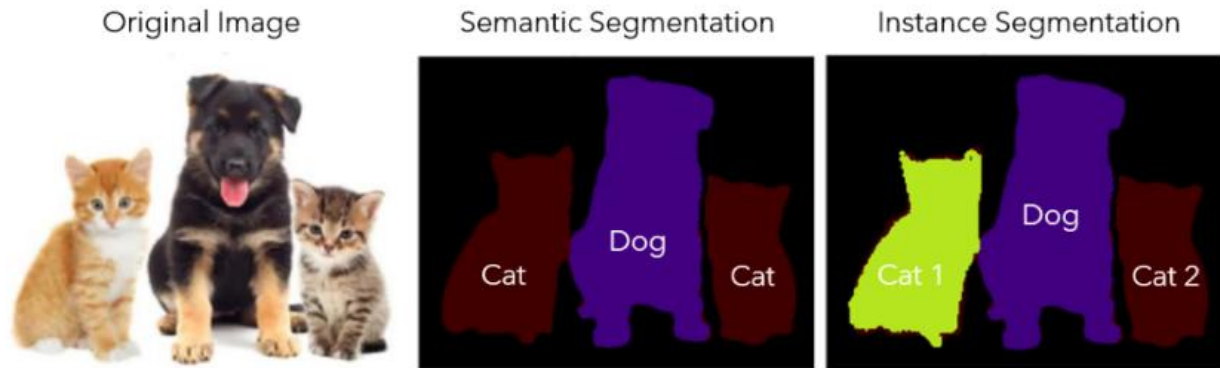
Fine-Tuning Object Detection

Using the Penn-Fudan Pedestrian Dataset

Introduction

In this lab, you will fine-tune a pre-trained Faster R-CNN model to detect pedestrians using the Penn-Fudan Pedestrian Detection dataset. This dataset is small (170 images), already annotated, and perfect for learning the fine-tuning workflow.

The dataset comes with **instance segmentation masks** for each pedestrian. We will convert these masks into bounding boxes automatically.



About the Penn-Fudan Dataset

Property	Details
Total Images	170 images of pedestrians on streets
Annotations	Pixel-level masks (PNG) for each person
Classes	1 class: Pedestrian (+ background)
Image Size	Varies (roughly 400×550 pixels)
Source	University of Pennsylvania & Fudan University
Format	Images (PNG) + Mask images (PNG)

What You Will Learn

- How to download and explore a ready-made detection dataset
- How to write a Dataset class that extracts bounding boxes from segmentation masks
- How to modify Faster R-CNN for a single-class detection task
- How to train the model and monitor the loss
- How to evaluate and visualize detections on test images
- How to save and reload your fine-tuned model

Install Libraries:

Open your terminal and install the required packages:

```
pip install torch torchvision Pillow matplotlib
pip install pycocotools
```

Verify your installation and check for GPU support:

```
import torch
print(f'PyTorch: {torch.__version__}')
print(f'CUDA: {torch.cuda.is_available()}')
```

Download the Penn-Fudan Dataset

Download and extract the dataset with a simple Python script:

```
import os
import urllib.request
import zipfile

# Download the dataset
url = ('https://www.cis.upenn.edu/~jshi/ped_html/'
      'PennFudanPed.zip')
zip_path = 'PennFudanPed.zip'

if not os.path.exists('PennFudanPed'):
    print('Downloading dataset...')
    urllib.request.urlretrieve(url, zip_path)
    with zipfile.ZipFile(zip_path, 'r') as z:
        z.extractall('.')
    os.remove(zip_path)
    print('Done!')
else:
    print('Dataset already exists.')
```

After extraction, the folder structure looks like this:

```
PennFudanPed/
├── PNGImages/           # 170 street photos
│   ├── FudanPed00001.png
│   ├── FudanPed00002.png
│   └── ...
└── PedMasks/           # Matching mask images
    ├── FudanPed00001_mask.png
    ├── FudanPed00002_mask.png
    └── ...
```

Explore the Data

Let's look at a sample image and its mask to understand the annotations:

```
from PIL import Image
import matplotlib.pyplot as plt
```

```
import numpy as np

# Load a sample image and its mask
img = Image.open(
    'PennFudanPed/PNGImages/FudanPed00001.png')
mask = Image.open(
    'PennFudanPed/PedMasks/FudanPed00001_mask.png')

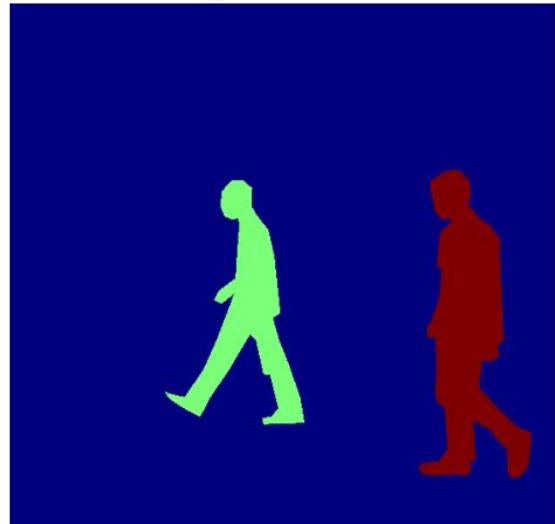
# Display side by side
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].imshow(img)
axes[0].set_title('Original Image')
axes[1].imshow(np.array(mask), cmap='jet')
axes[1].set_title('Segmentation Mask')
for ax in axes: ax.axis('off')
plt.tight_layout()
plt.show()

# Check unique values in the mask
mask_arr = np.array(mask)
print(f'Mask values: {np.unique(mask_arr)}')
print(f'0 = background, 1,2,3... = person IDs')
```

Original Image



Segmentation Mask



Build the Dataset Class

This is the key part. We write a PyTorch Dataset that loads each image and its mask, then extracts bounding boxes from the mask by finding the min/max coordinates of each person's pixels:

```
import os
import torch
import numpy as np
from PIL import Image
from torch.utils.data import Dataset
from torchvision import transforms

class PennFudanDataset(Dataset):
    """Penn-Fudan Pedestrian Detection Dataset.
    Loads images and extracts bounding boxes
    from segmentation masks automatically."""

    def __init__(self, root, transform=None):
        self.root = root
        self.transform = transform

        # List all images, sorted for consistency
        self.imgs = sorted(os.listdir(
            os.path.join(root, 'PNGImages')))
        self.masks = sorted(os.listdir(
            os.path.join(root, 'PedMasks')))

    def __len__(self):
        return len(self.imgs)

    def __getitem__(self, idx):
        # Load image
        img_path = os.path.join(
            self.root, 'PNGImages', self.imgs[idx])
        image = Image.open(img_path).convert('RGB')

        # Load mask
        mask_path = os.path.join(
            self.root, 'PedMasks', self.masks[idx])
        mask = np.array(
            Image.open(mask_path), dtype=np.int32)

        # Each unique nonzero value = one person
        obj_ids = np.unique(mask)
        obj_ids = obj_ids[obj_ids != 0] # remove bg

        # Extract bounding box for each person
```

```
boxes = []
for obj_id in obj_ids:
    pos = np.where(mask == obj_id)
    ymin = float(np.min(pos[0]))
    ymax = float(np.max(pos[0]))
    xmin = float(np.min(pos[1]))
    xmax = float(np.max(pos[1]))

    # Skip degenerate boxes
    if xmax <= xmin or ymax <= ymin:
        continue
    boxes.append([xmin, ymin, xmax, ymax])

num_objs = len(boxes)

# Convert everything to tensors
boxes = torch.as_tensor(
    boxes, dtype=torch.float32)
# All objects are class 1 (pedestrian)
labels = torch.ones(
    (num_objs,), dtype=torch.int64)
image_id = torch.tensor([idx])
area = (boxes[:, 3] - boxes[:, 1]) \
    * (boxes[:, 2] - boxes[:, 0])
iscrowd = torch.zeros(
    (num_objs,), dtype=torch.int64)

target = {
    'boxes': boxes,
    'labels': labels,
    'image_id': image_id,
    'area': area,
    'iscrowd': iscrowd,
}

if self.transform:
    image = self.transform(image)
else:
    image = transforms.ToTensor()(image)

return image, target
```

How Bounding Box Extraction Works

For each person in the mask (identified by a unique pixel value), we find the topmost, bottommost, leftmost, and rightmost pixels. These four coordinates form the bounding box.

Test the dataset to make sure it works:

```
dataset = PennFudanDataset('PennFudanPed')
print(f'Dataset size: {len(dataset)} images')

# Inspect first sample
img, target = dataset[0]
print(f'Image shape: {img.shape}')
print(f'Num people: {len(target["boxes"])}')
print(f'Boxes:\n{target["boxes"]}')
```

Create Data Loaders

Split the 170 images into training (140 images) and testing (30 images), then create DataLoaders:

```
from torch.utils.data import DataLoader

# Create full dataset
dataset = PennFudanDataset('PennFudanPed')

# Split into train and test
torch.manual_seed(42) # reproducible split
total = len(dataset)
train_size = 140
test_size = total - train_size

train_dataset, test_dataset = \
    torch.utils.data.random_split(
        dataset, [train_size, test_size])

print(f'Train: {len(train_dataset)} images')
print(f'Test: {len(test_dataset)} images')

# Custom collate (required for detection)
def collate_fn(batch):
    return tuple(zip(*batch))

train_loader = DataLoader(
    train_dataset,
    batch_size=4,
    shuffle=True,
    num_workers=2,
    collate_fn=collate_fn,
)
```

```
test_loader = DataLoader(
    test_dataset,
    batch_size=4,
    shuffle=False,
    num_workers=2,
    collate_fn=collate_fn,
)
```

Modify the Model

Load Faster R-CNN pre-trained on COCO and replace the classification head. We only need 2 classes: background (0) and pedestrian (1).

```
import torchvision
from torchvision.models.detection import \
    fasterrcnn_resnet50_fpn_v2, \
    FasterRCNN_ResNet50_FPN_V2_Weights
from torchvision.models.detection.faster_rcnn \
    import FastRCNNPredictor

def get_model(num_classes):
    # Load COCO pre-trained model
    weights = FasterRCNN_ResNet50_FPN_V2_Weights \
        .DEFAULT
    model = fasterrcnn_resnet50_fpn_v2(
        weights=weights)

    # Replace the classifier head
    in_features = model.roi_heads \
        .box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = \
        FastRCNNPredictor(in_features, num_classes)

    return model

# 2 classes: background + pedestrian
NUM_CLASSES = 2
model = get_model(NUM_CLASSES)

# Move to GPU if available
device = torch.device(
    'cuda' if torch.cuda.is_available()
    else 'cpu')
model.to(device)

print(f'Model on: {device}')
```

```
print(f'Classes: {NUM_CLASSES}')
```

What We Changed

The original Faster R-CNN classifies into 91 COCO categories. We replaced the **FastRCNNPredictor** head with one that outputs only 2 classes. The entire ResNet-50 backbone and Feature Pyramid Network keep their pre-trained weights — they already know how to extract useful visual features.

Configure the Optimizer

Set up SGD with momentum and a learning rate scheduler:

```
# Trainable parameters only
params = [p for p in model.parameters()
          if p.requires_grad]

optimizer = torch.optim.SGD(
    params,
    lr=0.005,
    momentum=0.9,
    weight_decay=0.0005,
)

# Reduce LR by 10x after epoch 5
lr_scheduler = torch.optim.lr_scheduler \
    .StepLR(optimizer, step_size=5, gamma=0.1)
```

Train the Model

```
import time

NUM_EPOCHS = 2

for epoch in range(NUM_EPOCHS):
    model.train()
    for images, targets in train_loader:
        images = [img.to(device) for img in images]
        targets = [{k: v.to(device) for k, v in t.items()} for t in targets]

        loss = sum(model(images, targets).values())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f'Epoch {epoch+1}/{NUM_EPOCHS} - loss: {loss.item():.4f}')

print('Training complete!')
```

Save and Load the Model

Save your trained model weights for future use:

```
# Save
save_path = 'pedestrian_detector.pth'
torch.save(model.state_dict(), save_path)
print(f'Saved to {save_path}')
```

```
# Load later
def load_model(path, num_classes, device):
    model = get_model(num_classes)
    model.load_state_dict(
        torch.load(path,
                    map_location=device,
                    weights_only=True))
    model.to(device)
    model.eval()
    return model
```

```
model = load_model(
    save_path, NUM_CLASSES, device)
print('Model reloaded!')
```

Test on Unseen Images

Run the fine-tuned model on test images and visualize the bounding boxes:

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from torchvision import transforms

def test_and_visualize(model, dataset,
                       device, num_images=4,
                       threshold=0.5):
    model.eval()
    fig, axes = plt.subplots(
        1, num_images, figsize=(5*num_images, 5))

    for i in range(num_images):
        img_tensor, target = dataset[i]
        with torch.no_grad():
            pred = model(
                [img_tensor.to(device)])[0]
```

```
# Filter by threshold
mask = pred['scores'] > threshold
boxes = pred['boxes'][mask].cpu()
scores = pred['scores'][mask].cpu()

# Convert tensor to displayable image
img_np = img_tensor.permute(
    1, 2, 0).numpy()

ax = axes[i] if num_images > 1 \
    else axes
ax.imshow(img_np)

for j in range(len(boxes)):
    x1, y1, x2, y2 = boxes[j].tolist()
    score = scores[j].item()
    rect = patches.Rectangle(
        (x1, y1), x2-x1, y2-y1,
        linewidth=2,
        edgecolor='lime',
        facecolor='none')
    ax.add_patch(rect)
    ax.text(x1, y1 - 5,
        f'Person {score:.0%}',
        color='white', fontsize=9,
        fontweight='bold',
        bbox=dict(
            facecolor='green',
            alpha=0.7))

ax.set_title(f'{len(boxes)} detected')
ax.axis('off')

plt.tight_layout()
plt.savefig('test_results.jpg', dpi=150)
plt.show()
print('Results saved to test_results.jpg')

# Run on test set
test_and_visualize(
    model, test_dataset, device)
```



Evaluate with Metrics

```
def evaluate_model(model, data_loader, device, threshold=0.5):
    model.eval()
    correct, predicted, actual = 0, 0, 0

    for images, targets in data_loader:
        images = [img.to(device) for img in images]

        with torch.no_grad():
            preds = model(images)

        for pred, target in zip(preds, targets):
            pred_boxes = pred['boxes'][pred['scores'] > threshold].cpu()
            gt_boxes = target['boxes']

            predicted += len(pred_boxes)
            actual += len(gt_boxes)

            for pb in pred_boxes:
                for gb in gt_boxes:
                    # IoU
                    x1, y1 = max(pb[0], gb[0]), max(pb[1], gb[1])
                    x2, y2 = min(pb[2], gb[2]), min(pb[3], gb[3])
                    inter = max(0, x2-x1) * max(0, y2-y1)
                    union = (pb[2]-pb[0])*(pb[3]-pb[1]) + (gb[2]-
gb[0])*(gb[3]-gb[1]) - inter
                    if inter / union > 0.5:
                        correct += 1
                        break

    precision = correct / max(predicted, 1)
    recall = correct / max(actual, 1)
    f1 = 2 * precision * recall / max(precision + recall, 1e-6)

    print(f'Precision: {precision:.1%} Recall: {recall:.1%} F1: {f1:.1%}')

# Run evaluation
print('\nEvaluation Results')
```

```
evaluate_model(model, test_loader, device)
```

Understanding the Metrics

Metric	What It Means
Precision	Of all pedestrians the model detected, what percentage were real? (Low false positives = high precision)
Recall	Of all actual pedestrians in the images, what percentage did the model find? (Low missed detections = high recall)
F1 Score	The balanced average of precision and recall. Above 80% is good for this dataset.

Lab Task:

1. **Practice:** Practice the above example.
2. **Threshold tuning:** Run `evaluate_model` with thresholds 0.3, 0.5, 0.7, and 0.9. How does precision vs. recall change? Plot a precision-recall curve.
3. **Train for more epochs:** Try 15 and 20 epochs.
4. **Smaller training set:** Use only 50 or 80 training images instead of 140. How much does accuracy drop?
5. **Freeze the backbone:** Add this before training: `for p in model.backbone.parameters(): p.requires_grad = False`. And test.
6. **Batch size experiment:** Try batch sizes of 1, 2, 4, and 8. Which gives the best balance of speed and stability?
7. **Try a lighter model:** Replace the model with `fasterrcnn_mobilenet_v3_large_fpn` from `torchvision`.